

GlobalTech HR Attrition Solution Guide

GlobalTech HR Attrition Dashboard — Full Solution & Explanation Guide

Every step, every formula, broken down piece by piece — including WHY each decision was made

Project Overview

Purpose of This Project — What We Were Trying to Find

PHASE 0: The Audit — What We Looked For and Why

PHASE 2: Cleaning — Full List with Reasoning

PHASE 3: Data Modeling — Why Only 3 Tables

The 3 tables built:

Why there's no separate "Employees" table

Why Departments still got its own table (even though it's simple)

Why Calendar exists (same reasoning as any project)

Quick Reference: Why These 2 Extra Tables (Departments + Calendar)

PHASE 4: DAX Measures — Why These 6, and Full Formula Breakdowns

Why these specific measures were chosen (not just "these are common ones")

Measure 1: Total Employees

Measure 2: Total Attritions

Measure 3: Attrition Rate

Measure 4: Average Salary

Measure 5: Average Satisfaction Score

Measure 6: Average Tenure (Years) — the most complex measure in this project

Why We Picked These Exact Charts

PHASE 5: Report Layout — What Was Built and Why

PHASE 5: Report Build — What Was Built, What Went Wrong, and Why

Final report structure

Issue 1: KPI Cards showing raw values instead of intended ones

Issue 2: Line chart displaying months in the wrong order

Issue 3: Redundant/confusing auto-generated chart titles

Final verified numbers from the built report

Why the insight callout was worded the way it was

Key Takeaways From This Project

GlobalTech HR Attrition Dashboard — Full Solution & Explanation Guide

Every step, every formula, broken down piece by piece — including WHY each decision was made

Project Overview

Dataset: GlobalTech_HR_RAW.xlsx — 446 raw rows of employee data, intentionally messy. **Final verified numbers:** - Total Employees: 430 - Total Attritions: 95 - Attrition Rate: 22% - Average Salary: \$87,760 - Average Satisfaction Score: 3.01 - Average Tenure: 3.02 years

Purpose of This Project — What We Were Trying to Find

The scenario, in plain terms: GlobalTech Solutions is a made-up company that's losing employees, and nobody knows exactly why. We had messy employee records, but no clear picture of where the problem was worst, who was most likely to leave, or whether there was a warning sign we could actually track.

The simple questions we wanted answered: 1. How bad is the problem — one clear number? 2. Is it happening everywhere equally, or mostly in one department or job? 3. Do unhappy employees actually leave more, or is that just a guess? 4. How long do people usually stay? 5. Does hiring speed up or slow down at certain times of year?

What we actually found: - **22% of employees leave** — about 1 in every 5 people. - It's not spread evenly. **Sales has the worst attrition (24.6%)**. Finance and Marketing keep their people the longest. - Job title matters even more than department. **Software Engineers leave at a 43% rate** — way higher than any other role. - Happier employees do stay longer, but the difference is small: people who left rated their satisfaction **2.92 out of 5**, and people who stayed rated it **3.04 out of 5**. - The average employee sticks around for **3 years**.

Why this project is worth showing off: it turns a vague worry ("we're losing people, why?") into real, specific numbers someone could actually act on. It also shows a good habit — checking one level deeper (job role, not just department) is what uncovered the biggest finding (the 43% Software Engineer number), instead of stopping at the first easy answer.

PHASE 0: The Audit — What We Looked For and Why

Before opening Power Query, the raw file was scanned for: - Where the real table starts (title/merged rows often push headers down) - Numbers stored as text (Salary — mixed \$85,000, 92000 USD, plain numbers) - Same real value spelled differently (Department, Gender, Education, Marital Status, OverTime, Work Mode) - Meaningful blanks (Satisfaction Survey — unanswered surveys, NOT missing/bad data) - Duplicate and blank rows

Why this step matters: Auditing first means you fix problems in a planned order instead of discovering them one at a time mid-clean, which is slower and more error-prone.

PHASE 2: Cleaning — Full List with Reasoning

Column	Fix Applied	Why
Header rows	Remove Top Rows (2), then Use First Row as Headers	Title/spacer row was pushing real headers to row 3
Department	Replace Values per spelling variant → 6 clean values	Fragmented spellings ("sales"/"SALES"/"Sales") would create false-separate categories in any chart
	Replace M→Male, F→Female,	

Gender	male→Male → 2 clean values	Same fragmentation risk
Education	Capitalize Each Word → 4 clean values	Quick automated fix works for most casing issues
Marital Status	Capitalize Each Word → 3 clean values	Same
OverTime	Replace Y→Yes, N→No, yes→Yes → 2 clean values	Inconsistent short/long forms needed manual mapping
Work Mode	Capitalize Each Word → 3 clean values	Same automated fix
Employee Name	Trim + Capitalize Each Word	Removes invisible whitespace that makes two identical-looking names register as different values
Annual Salary	Change Type→Text (undo auto-error) → Replace Values (\$, USD → nothing) → Change Type→Whole Number	Classic “fix value before type” — converting first throws hard errors on unparseable text
Satisfaction Survey	Left untouched, blanks preserved	Blanks represent real non-responses. Filling them with a fake number (like 0 or the average) would corrupt any AVERAGE() calculation later — this is a case where “doing nothing” is the correct, deliberate choice
Hire Date / Exit Date	Cleaned to Date type; Exit Date blanks preserved	Blank Exit Date = still employed. This is meaningful information, not missing data
Blank rows	Filtered via EmployeeID → Remove Empty	Safer than manual deletion; catches rows a custom column might otherwise hide from Remove Blank Rows
Duplicates	Select all columns → Remove Duplicates	Selecting all columns ensures only TRUE full-row duplicates are removed, not legitimately similar-but-different rows

Result: 430 clean rows, 16 columns, verified against multiple independent checks (filter dropdown counts, DISTINCTCOUNT measures).

PHASE 3: Data Modeling — Why Only 3 Tables

This is worth explaining properly, because it’s a different structure than a typical sales/orders project (like NorthStar), and understanding *why* is the actual lesson here.

The 3 tables built:

1. **HR Export** (fact table — one row per employee)
2. **Departments** (dimension table)
3. **Calendar** (dimension table)

Why there's no separate "Employees" table

In a sales dataset like NorthStar, one customer places MANY orders — so the raw data has that customer's name repeated across many rows, and you extract a clean Customers table to remove that repetition (this is called normalizing denormalized data).

In this HR dataset, that repetition doesn't exist. Each row in HR Export already represents exactly ONE employee, ONE time. There's no "employee placing 5 different HR records" scenario — an employee's Age, Salary, Department, etc. is a single fixed set of facts per person. Since there's nothing to de-duplicate or extract, HR Export can safely act as BOTH your fact table and your (only) real dimension for employee-level facts. Building a separate "Employees" table here would just be an identical copy of HR Export with extra steps — no actual benefit.

The general rule this teaches: you only extract a separate dimension table when the SAME real-world entity repeats across multiple rows in your raw data. If each row already IS one unique entity, there's nothing to extract.

Why Departments still got its own table (even though it's simple)

Department names appear repeated across many employee rows (many employees share "Sales" as their department) — this DOES fit the repetition pattern, so it was worth extracting into its own dimension table, even though it only has one column. This also sets you up cleanly if you ever wanted to add more department-level facts later (like a department budget or head-of-department name) without cluttering HR Export.

Why Calendar exists (same reasoning as any project)

SUMMARIZE on Hire Date alone would only capture dates where a hire actually happened, leaving gaps on any day with zero hires — which breaks trend charts and month-over-month comparisons. A generated CALENDAR() table guarantees a complete, unbroken date range, matching every prior project's reasoning.

Quick Reference: Why These 2 Extra Tables (Departments + Calendar)

Table	Extracted because...	Row count check
Departments	The value repeats across MANY employee rows (multiple employees share "Sales", "Engineering", etc.) — this repetition is exactly what makes a value worth pulling into its own dimension table Not extracted from existing data at all —	6 rows, verified against <code>DISTINCTCOUNT('HR Export'[Department])</code>

Calendar

GENERATED fresh to guarantee a complete, unbroken date range (including days with zero hires), which SUMMARIZE on Hire Date alone could never provide

Full Jan 2018-Dec 2024 range, one row per day

The one-line rule these two tables both follow: extract a table when a value repeats across rows (Departments) OR when you need guaranteed date completeness for time-based analysis (Calendar) — never just to “have more tables” for its own sake. HR Export itself needed neither treatment, since it has no repeating entity to extract — see the full explanation below.

Only one ACTIVE relationship is allowed between any two tables. Since this dashboard’s core story is about *hiring trends and workforce composition over time* (not primarily “when did people leave”), Hire Date was chosen as the active connection. Exit Date could still be connected as a second, inactive relationship if deeper “time-to-attrition” analysis was ever needed later, activated per-measure with USERELATIONSHIP().

PHASE 4: DAX Measures — Why These 6, and Full Formula Breakdowns

Why these specific measures were chosen (not just “these are common ones”)

Every measure here maps directly to a specific business question an HR stakeholder would actually ask:

Measure	Business Question It Answers
Total Employees	“How big is our workforce?”
Total Attritions	“How many people have we lost?”
Attrition Rate	“What % of our workforce leaves — the single most important HR health metric”
Average Salary	“Are we paying competitively / consistently?”
Average Satisfaction Score	“Are people generally happy here?”
Average Tenure	“How long do people typically stay?”

No measure was added just because it was easy to write — each one was chosen because it’s a number a real HR/People Analytics team tracks. A dashboard full of measures nobody would actually ask for is a common beginner mistake; every measure here earns its place.

Measure 1: Total Employees

Total Employees =
DISTINCTCOUNT('HR Export'[EmployeeID])

Piece by piece: - DISTINCTCOUNT(...) — counts how many UNIQUE values exist in a column, ignoring repeats - 'HR Export'[EmployeeID] — the specific column being counted

Why DISTINCTCOUNT instead of plain COUNT: If any duplicate EmployeeID slipped through cleaning undetected, COUNT would inflate the total. DISTINCTCOUNT protects against that automatically — a small but meaningful safety habit.

Measure 2: Total Attritions

Total Attritions =
CALCULATE(DISTINCTCOUNT('HR Export'[EmployeeID]), 'HR Export'[Attrition] = "Yes")

Piece by piece: - DISTINCTCOUNT('HR Export'[EmployeeID]) — same counting logic as Measure 1 - CALCULATE(..., condition) — CALCULATE lets you inject an extra filter condition directly into a measure. Normally this measure would count ALL employees; wrapping it in CALCULATE with a condition narrows what gets counted - 'HR Export'[Attrition] = "Yes" — the actual filter condition: only count rows where the Attrition column equals the text "Yes"

Why CALCULATE specifically: DAX has no built-in "COUNT IF" function the way Excel does — CALCULATE is how DAX achieves the same conditional counting behavior. This is one of the most common DAX patterns you'll reuse in almost every future project.

Measure 3: Attrition Rate

Attrition Rate =
DIVIDE([Total Attritions], [Total Employees], 0)

Piece by piece: - DIVIDE(numerator, denominator, fallback) — DAX's safe division function - [Total Attritions] — the numerator: how many left - [Total Employees] — the denominator: total headcount - 0 — what to display if the denominator is ever zero (protects against a blank/error if a filtered view somehow has zero employees)

Why DIVIDE instead of the plain / operator: the plain / would throw a hard error the moment any filter context produces a zero denominator (for example, filtering to a department with literally zero employees). DIVIDE fails safely instead, returning your chosen fallback value.

Why this measure references two OTHER measures instead of recalculating from scratch: Measures can call other measures — this keeps your formulas short, readable, and consistent. If you ever needed to change how "Total Employees" is calculated, you'd only update it in ONE place, and Attrition Rate would automatically stay correct.

Measure 4: Average Salary

Average Salary =
AVERAGE('HR Export'[Annual Salary])

Piece by piece: - AVERAGE(column) — a built-in DAX function that sums a column and divides by the count of non-blank values automatically

Why this one is simple (no CALCULATE, no SUMX needed): Annual Salary is already a clean, single number per row with no per-row math required first (unlike NorthStar's Total Revenue, which needed Qty × Price × Discount calculated row-by-row before summing). When your data is already in the exact shape you need, plain AVERAGE() is the correct, simplest tool — using something more complex here would just be unnecessary complexity.

Measure 5: Average Satisfaction Score

Average Satisfaction Score =
AVERAGE('HR Export'[Satisfaction Survey])

Piece by piece: Identical structure to Measure 4 — just pointed at a different column.

Why this measure ONLY works correctly because of a Phase 2 decision: AVERAGE() automatically SKIPS blank/null values — it does not count them as zero. This is exactly why the blank survey responses were left untouched in Phase 2 instead of being filled with a placeholder number. If those blanks had been filled with 0 during cleaning, this measure would have been silently wrong — dragging the average down artificially by including “non-answers” as if they were low scores. This is a good example of how a Phase 2 decision directly determines whether a Phase 4 measure is trustworthy.

Measure 6: Average Tenure (Years) — the most complex measure in this project

Average Tenure (Years) =
AVERAGEX(
 'HR Export',
 DATEDIFF('HR Export'[Hire Date],
 IF(ISBLANK('HR Export'[Exit Date]),DATE(2024,12,31),
 'HR Export'[Exit Date]),
 YEAR)
)

Piece by piece, working from the inside out:

1. **ISBLANK('HR Export'[Exit Date])** — checks, for the CURRENT row being processed, whether Exit Date is empty. Returns TRUE if the employee never left (still employed).
2. **IF(ISBLANK(...), DATE(2024,12,31), 'HR Export'[Exit Date])** — this is the core logic: *“If this employee is still employed, use Dec 31, 2024 as their end-point for the calculation instead. If they DID leave, use their actual Exit Date.”* This lets one formula correctly handle both employed and departed staff without needing two separate measures.
3. **DATEDIFF(startdate, enddate, YEAR)** — calculates the whole-number difference between two dates, in years. Takes Hire Date as the start, and whichever end-point the IF statement produced.
4. **AVERAGEX('HR Export', <expression>)** — this is the outer wrapper that makes the whole thing work across the whole table. AVERAGEX runs the entire DATEDIFF/IF/ISBLANK calculation SEPARATELY for every single row in HR Export, producing one tenure-in-years number per employee, THEN averages all of those results together.

Why AVERAGEX instead of plain AVERAGE: Plain AVERAGE() can only average an EXISTING column — it can't perform a calculation first. Since “tenure” isn't a column that already exists in your data (it has to be calculated from two other date columns, with conditional logic), you need the “X” version of the function, which calculates something new per row before averaging. This is the same underlying pattern as SUMX from NorthStar's Total Revenue measure — any time you need row-by-row math before aggregating, reach for the X-suffix version of the function.

Why DATE(2024,12,31) instead of TODAY(): This was a live bug caught during building — the first version used TODAY(), which returns the actual current calendar date (2026 at the time of building), making tenure calculations balloon to include 2 extra years for anyone still

employed. Since this dataset was designed around a 2024 timeframe, hardcoding DATE(2024,12,31) keeps the tenure calculation anchored correctly and prevents the number from silently increasing every time someone reopens the file in a later year — important for a portfolio piece meant to stay accurate indefinitely.

Why We Picked These Exact Charts

Before the chart list — here’s the simple rule we followed: **pick whatever chart makes the answer obvious at a glance, without making someone do math in their head.**

- One important number (Total Employees, Attrition Rate) → **Card**. Nothing to compare it to, so just make it big and easy to spot.
- Comparing departments or job roles against each other → **Bar chart, biggest to smallest**. Bars are the easiest way for eyes to compare sizes.
- Checking if satisfaction and attrition are connected → **Two bars side by side (Yes vs No)**. This is a straight “compare two things” question, so two bars answer it fastest.
- Hiring activity over the months → **Line chart**. Lines are the best way to show a trend going up or down over time.

PHASE 5: Report Layout — What Was Built and Why

Visual	Fields Used	Why This Visual Type
4 KPI Cards	Total Employees, Attrition Rate, Average Tenure, Average Salary	Single standalone facts — Cards give maximum visual weight with zero clutter, anchoring the page
Bar chart (sorted desc)	Department (axis) × Attrition Rate (value)	Category comparison — bar length lets the eye instantly rank departments by risk
Bar chart (sorted desc)	Job Role (axis) × Attrition Rate (value)	Same reasoning, different category cut
Bar/scatter	Attrition (Yes/No) × Average Satisfaction Score	Tests a relationship between two measures — directly answers “does low satisfaction predict leaving?”
Line chart	Month Name (from Calendar, sorted by Month Number) × hire count	Trends over time need continuity — a line shows hiring patterns/seasonality that a bar chart would present more chopply
Slicers	Department, Work Mode, OverTime	Lets a viewer interactively explore instead of only seeing one fixed view

Why Attrition Rate got the visually largest/boldest card: It’s the single metric this entire dashboard exists to explain — every other visual on the page exists to help answer “why is Attrition Rate what it

is,” so it earns the most prominent visual position.

PHASE 5: Report Build — What Was Built, What Went Wrong, and Why

This section documents the actual build process, including two real issues hit along the way — both are common, both are worth understanding rather than just memorizing the fix.

Final report structure

- **Title/Subtitle:** “GlobalTech Solutions — Workforce & Attrition Dashboard” / “Employee retention, satisfaction, and department trends | 2018–2024”
- **4 KPI Cards:** Total Employees (430), Attrition Rate (22%), Average Tenure (3.02 years), Average Salary (\$87.76K)
- **Attrition Rate by Department** (bar chart, sorted descending)
- **Attrition Rate by Job Role** (bar chart, sorted descending)
- **Average Satisfaction Score by Attrition** (bar chart, Yes vs No comparison)
- **Total Employees by Month Name** (line chart, using Calendar table)
- **3 Slicers:** Department, Work Mode, OverTime
- **Insight callout text box** stating the satisfaction finding

Issue 1: KPI Cards showing raw values instead of intended ones

What happened: Two cards initially showed incorrect output: - Attrition Rate displayed as 0.22 instead of 22% - The 4th card showed 38M – Sum of Annual Salary instead of the intended Average Salary measure

Why the Attrition Rate issue happened: DAX always stores a percentage-style measure as a raw decimal internally (0.22) — the % symbol is purely a DISPLAY format layered on top, not part of the calculation itself. A newly created measure defaults to General number formatting until you explicitly set it to Percentage via Modeling tab → Formatting → Percentage.

Why the Salary card issue happened: Dragging the raw Annual Salary COLUMN directly onto a card defaults to a SUM() aggregation automatically — Power BI has to pick some default aggregation for a raw numeric column, and Sum is its default guess. This produced a technically-correct-but-unintended total (the sum of every salary in the company, \$38M) instead of the actual Average Salary MEASURE that had already been built in Phase 4. The fix was replacing the raw column with the purpose-built measure.

The general lesson: always double check whether a card is pulling a raw COLUMN (which needs you to pick/verify an aggregation) versus an already-built MEASURE (which has its calculation logic locked in) — these look similar when dragged onto a visual but behave very differently.

Issue 2: Line chart displaying months in the wrong order

What happened: The “Total Employees by Month Name” line chart initially displayed months in a scrambled, non-calendar order (June, October, May, January...) instead of January through December.

First fix attempted — Sort by Column at the table level:

Data view → Calendar table → Month Name column → Modeling tab → Sort by Column → Month Number

This is the CORRECT standard fix and should work in most cases — it tells Power BI “whenever Month Name is used anywhere in the model, secretly sort it using Month Number’s order instead of alphabetical order.”

Why it didn’t immediately fix the chart: Individual visuals can carry their OWN independent axis sort override that takes priority over the table-level Sort by Column setting. This is a lesser-known Power BI behavior — the table-level sort is a strong default, but a specific chart can still have a stale or manually-set sort rule from earlier in its creation that ignores it.

Actual fix that worked: Clicking the “...” (more options) menu directly on the chart itself → Sort axis → re-selecting the correct sort field forced the visual to re-evaluate and pick up the table-level Month Number ordering correctly.

The general lesson: if Sort by Column doesn’t seem to be working, check the individual VISUAL’s own sort settings before assuming the table-level configuration is wrong — the two are separate settings that can conflict.

Issue 3: Redundant/confusing auto-generated chart titles

What happened: Charts auto-named themselves things like “Attrition Rate by Department by Department” and “Attrition Rate by Department by Job Role.”

Why this happened: Power BI auto-generates a chart’s title by combining the MEASURE’s own name with whatever field is on the axis. Since the measure itself was named “Attrition Rate by Department” (a naming choice made back in Phase 4, intended to be reused across multiple charts with different axis fields), any chart using that measure inherited that full name as a prefix, then appended the axis field name again — creating redundancy.

Fix: Manually overriding each chart’s title text via Format pane → Title, rather than relying on the auto-generated name.

The lesson for future projects: if you plan to reuse the same measure across multiple charts with different axis breakdowns (a common, correct DAX practice), expect to manually clean up chart titles afterward — the auto-naming doesn’t anticipate this reuse pattern.

Final verified numbers from the built report

Chart	Finding
Attrition Rate by Department	Sales highest (24.6%), Marketing lowest (19.4%)
Attrition Rate by Job Role	Software Engineer highest by a wide margin (43%)
Satisfaction by Attrition	Stayed = 3.04 average, Left = 2.92 average
Hiring trend	Fluctuates month to month, no single dominant seasonal pattern

Why the insight callout was worded the way it was

The chosen callout — *“Employees who left report a lower average satisfaction score (2.92) compared to those who stayed (3.04) — a modest but measurable early warning signal”* — deliberately uses the words “modest” and “measurable” rather than overstating the finding. The actual gap (0.12 points on a 5-point scale) is real but not dramatic. Accurately describing the SIZE of a finding, not just its direction, is what separates credible analysis from an exaggerated claim — and is worth doing consistently in any portfolio piece, since overstating small findings is a common way beginner analysis loses credibility with an experienced reviewer.

Worth noting for future iteration: the Software Engineer attrition rate (43%) is a substantially larger and more attention-grabbing finding than the satisfaction gap, and could arguably be the LEAD insight instead of or alongside the satisfaction one — a good example of how the “best” insight to highlight isn’t always the first one you calculate, and it’s worth comparing the relative size of multiple findings before deciding which one anchors your summary.

Key Takeaways From This Project

1. **Not every project needs the same number of dimension tables** — the right number depends on whether entities repeat in your raw data, not on following a fixed template. HR data needed fewer tables than order data because employees don’t naturally repeat the way customers placing multiple orders do.
2. **CALCULATE is DAX’s way of doing conditional counting/summing** — there’s no separate “COUNTIF” function; CALCULATE plus a condition is the standard pattern.
3. **The X-suffix functions (SUMX, AVERAGEX) exist specifically for row-by-row calculations before aggregating** — use them whenever the value you need doesn’t already exist as a plain column.
4. **A Phase 2 cleaning decision can make or break a Phase 4 measure’s accuracy** — the Satisfaction Survey blanks are a direct example: leaving them null was what made AVERAGE() work correctly later.
5. **Hardcode reference dates for portfolio pieces instead of using TODAY()** when the dataset represents a fixed historical period — otherwise your numbers silently drift every time the file is reopened in a future year.
6. **Card visuals behave differently depending on whether you drag a raw column or a pre-built measure onto them** — a raw column defaults to an aggregation guess (usually Sum), while a measure carries its own locked-in calculation logic. Always verify which one you actually dragged.
7. **Chart-level sort settings can override table-level Sort by Column** — if a chart still displays in the wrong order after setting Sort by Column correctly, check the individual visual’s own axis sort settings before assuming the table configuration failed.
8. **When reusing one measure across multiple charts with different axis breakdowns, expect to manually fix auto-generated chart titles** — Power BI’s auto-naming doesn’t anticipate this reuse pattern and will produce redundant titles.
9. **State the actual SIZE of a finding, not just its direction, in any insight callout** — a real but modest gap described honestly (using words like “modest,” “slight,” etc. where accurate) is more credible than an overstated claim, and builds trust with anyone reviewing your work.